

```

import pygame
import sys
import sqlite3
import constants
import random                # importing libraries and other classes
import map_list
import math
import time

pygame.init()    # initialising pygame
screen = pygame.display.set_mode((constants.screen_width,
constants.screen_height)) # setting screen size and caption
pygame.display.set_caption("Anas' Abyss of Shadows") # setting games name
transparent_background = pygame.Surface((constants.screen_width - 200,
constants.screen_height - 200)) # setting background for pause menu
transparent_background.fill((0,0,0, 128))
font = pygame.font.Font(None, 36) # setting main font

clock = pygame.time.Clock()    # control fps

current_username = None # assigning the global username variable to none so that
the current players username can be accessed

# creating the database players table
conn = sqlite3.connect('game.db')
cursor = conn.cursor()

cursor.execute("""CREATE TABLE IF NOT EXISTS players (
                player_id INTEGER PRIMARY KEY,
                username TEXT NOT NULL,
                password TEXT NOT NULL,
                score INTEGER,
                waves INTEGER,
                time INTEGER
                )""")

conn.commit()

# creating functions

def check_player_username(username):
    conn = sqlite3.connect('game.db')                # code for checking username
exists in the database
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM players WHERE username = ?", (username,)) #
fetches all record of current username entered
    data = cursor.fetchall()
    conn.close()

    if len(data) > 0:                # returns false if they dont enter anything
        return True
    else:

```

```

        return False

def check_player_password(username, password):
    conn = sqlite3.connect('game.db') # code for
    # checking users password matches the password they entered
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM players WHERE username = ? AND password = ?",
    (username, password)) # fetches record for current username entered to check their
    # password
    data = cursor.fetchall()
    conn.close()

    if len(data) > 0: # returns false if they dont enter anything
        return True
    else:
        return False

def add_player(username, password, score, waves, time):
    try: # code
    # for adding a player into the database after creating an account
        conn = sqlite3.connect('game.db')
        cursor = conn.cursor()
        cursor.execute("INSERT INTO players (username, password, score, waves,
time) VALUES (?, ?, ?, ?, ?)", (username, password, score, waves, time))
        conn.commit() # inserts all the
    # data into the database for an account successfully created
    except sqlite3.Error as e:
        print("error occured:", e) # checks for error

    conn.close()

def update_stats(username, score, waves, time):
    cursor.execute("SELECT score FROM players WHERE username = ?", (username,))
    # code to update users stats whenever it increases
    score_data = cursor.fetchall()
    if score_data:
        current_score = score_data[0][0] # if the score
    # value is not empty fetches it
        if score > current_score: # if the users
    # current score is greater than their highest score in the database, it updates it
            cursor.execute("UPDATE players SET score = ? WHERE username = ?",
(score, username))

        cursor.execute("SELECT waves FROM players WHERE username = ?", (username,)) #
    # selects waves record where the username is equal to the accounts username and if
    # it is greater their current waves greater it is updated in the database
        waves_data = cursor.fetchall()
        if waves_data:
            current_waves = waves_data[0][0] # if the users current
    # wave is greater than their highest wave in the database, it updates it
            if waves > current_waves:

```

```

        cursor.execute("UPDATE players SET waves = ? WHERE username = ?",
(waves, username))

    cursor.execute("SELECT time FROM players WHERE username = ?", (username,))
    time_data = cursor.fetchall()
    if time_data:
        current_time = time_data[0][0] # if the users current
time is greater than their highest time survived in the database, it updates it
        if time > current_time:
            cursor.execute("UPDATE players SET time = ? WHERE username = ?",
(time, username))

    conn.commit()

def search_player(): # code to enter searched player and output their stats

    conn = sqlite3.connect('game.db')
    cursor = conn.cursor()
    player = input("Enter players stats you want to search: ")
    cursor.execute("SELECT username FROM players")
    rows = cursor.fetchall()

    usernames = [row[0] for row in rows] # gets all the usernames in the
database

    ordered_usernames = sorted(usernames) # orders them so they can be used in a
binary search
    searched_user = binary_search(ordered_usernames,player)

    if searched_user is not None: # if the searched user was found it
fetches there stats and returns it otherwise it returns false
        cursor.execute("SELECT * FROM players WHERE username = ?",
(ordered_usernames[searched_user],))
        player_stats = cursor.fetchall()
        return player_stats
    else:
        return False

def leaderboard():
    player_stats = None
    running = True
    while running:

        font = pygame.font.Font(None, 36) #setting fonts
        font_title = pygame.font.Font(None, 48)

        leaderboard_rect = pygame.Rect(100,100,constants.screen_width - 200,
constants.screen_height - 200)
        pygame.draw.rect(screen,(0,0,0), leaderboard_rect)
        leaderboard_rect = font.render("name:             most kills:
highest wave:         highest time:", True, (255,255,255))

```

```

        screen.blit(leaderboard_rect, (225,175))    #setting titles rect

        title_text = font_title.render("Leaderboard", True, (255,255,255))

        screen.blit(title_text, (constants.screen_width //2 - 100, 110))
# writing leaderboard text on screen

        back_rect = pygame.Rect(100,100,100,50)
        pygame.draw.rect(screen,(128,128,128), back_rect)
        back_surface = font.render("Back", True, (0,0,0))
        back_rect = back_surface.get_rect(center=back_rect.center)    #
return back from leaderboard
        screen.blit(back_surface, back_rect)

        search_rect = pygame.Rect(150,600,170,50)
        drawing_text(screen,"Search Player:",font,(255,255,255),None,None,
(128,128,128),search_rect) # drawing search button

        cursor.execute("SELECT username, score, waves, time FROM players ORDER BY
score DESC LIMIT 6") # displays top 6 players in order of kills
        data = cursor.fetchall()

        def converting_time(time):
            minutes = int(time // 60)
            seconds = int(time % 60)
            time_string = f"{minutes} mins {seconds} secs"    #
function to convert time from seconds to minutes.seconds format
            return time_string

        y_position = 235

        for username, score, waves, time in data:
            time_string = converting_time(time)
            if username == current_username:
                name = font.render(f"{username}", True, (255,215,0))
# if they are logged in it will show the top players stats by fetching the data
from database
                screen.blit(name, (225, y_position))
# using variable y position and increasing it after each user and highlight their
current account in yellow
                score = font.render(f"{score}", True, (255,215,0))
                screen.blit(score, (425,y_position))
                waves = font.render(f"{waves}", True, (255,215,0))
                screen.blit(waves, (625,y_position))
                time = font.render(time_string, True, (255,215,0))
                screen.blit(time, (775,y_position))

            else:
                name = font.render(f"{username}", True, (255,255,255))
# if they are not logged in it will present the same just no account will be
yellow
                screen.blit(name, (225, y_position))
                score = font.render(f"{score}", True, (255,255,255))

```

```

        screen.blit(score, (425,y_position))
        waves = font.render(f"{waves}", True, (255,255,255))
        screen.blit(waves, (625,y_position))
        time = font.render(time_string, True, (255,255,255))
        screen.blit(time, (775,y_position))
        y_position += 60

    for event in pygame.event.get():                # checking events
if clicked back button or search button
        mouse_pos = pygame.mouse.get_pos()
        if event.type == pygame.MOUSEBUTTONDOWN:
            if back_rect.collidepoint(mouse_pos):
                running = False
            elif search_rect.collidepoint(mouse_pos):
                player_stats = search_player()
        elif event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

    if player_stats:
        name = player_stats[0][1]                    # if players stats is not empty
it fetches the name, kills and waves of the searched player and displays it
        kills = player_stats[0][3]
        waves = player_stats[0][4]
        drawing_text(screen,f"name: {name}    kills: {kills}    waves:
{waves}",font,(255,255,255),constants.screen_width//2+100,630)
        elif player_stats == False:
            drawing_text(screen,"Player not found",font,
(255,255,255),constants.screen_width//2+100,630) # otherwise saying player not
found

    pygame.display.flip()

def drawing_text(screen, text, font, colour, x=None , y=None, rect_colour=None,
rect_dimension=None):    # function for drawing text on the screen with or
without a rect during the main game
    # if it has x and y coordinates they are set otherwise they are set to
    None and if they have a rect, dimensions and colour are set otherwise set to None
    if rect_dimension is not None:    # if it has a visible rect around it
        rect_dimension.inflate_ip(6,6) # increases the default size
        pygame.draw.rect(screen,rect_colour,rect_dimension)
        text = font.render(text,True,colour,rect_colour)
        text_rect = text.get_rect(center=rect_dimension.center)
    else:
        text = font.render(text,True,colour)    # if it is just text without
rect around it
        text_rect = text.get_rect()
        if x is not None and y is not None:    # if it has a x and y
coordinate
            text_rect.center = (x,y)

    screen.blit(text, text_rect)

```

```

        return text_rect # returns the rect of the text to be
used for mouse collision

def binary_search(list, wanted_user): # binary search used to search for
an item in a list (usernames in the database)
    lower = 0
    upper = len(list) - 1 # sets the upper pointer to the last item in the
list

    while lower <= upper: # until the lower is greater than or equal to the
upper pointer, which means the item has been found or its not in the list, the
code will execute
        midpoint = (upper + lower)//2 # finds midpoint of each loop
        current_user = list[midpoint]

        if current_user == wanted_user: # if it is found username returned
otherwise if current username is less than wanted it sets lower pointer to one
above it otherwise sets upper to one below
            return midpoint
        elif current_user < wanted_user:
            lower = midpoint + 1
        else:
            upper = midpoint - 1

    return None # returns none if not found

def controls(pause):
    running = True
    while running:
        transparent_background = pygame.Surface((constants.screen_width - 200,
constants.screen_height - 200))
        transparent_background.fill((0,0,0, 128))
        transparent_rect =
transparent_background.get_rect(center=screen.get_rect().center) #
setting the background of leaderboard
        screen.blit(transparent_background, transparent_rect)

        font = pygame.font.Font(None, 36)
        font_title = pygame.font.Font(None, 48)

        title_text = font_title.render("Leaderboard", True, (0,0,0))

        screen.blit(title_text, (constants.screen_width //2 - 100, 110))
# writing leaderboard text on screen

        back_rect = pygame.Rect(100,100,100,50)
        pygame.draw.rect(screen,(128,128,128), back_rect)
        back_surface = font.render("Back", True, (0,0,0))
        back_rect = back_surface.get_rect(center=back_rect.center) #
return back from controls

```

```

screen.blit(back_surface, back_rect)

# setting a list of strings to display on how to play screen
all_text = [
    "To move use WASD.",
    "To shoot use left click.",
    "Kill as many enemies as you can and survive as many waves",
    "while using health packs and power ups to your advantage.",
    "Play as a guest or create an account to save your stats",
    "and compete on the leaderboard!"
]

if pause is not None:
    for distance, text in enumerate(all_text):
        drawing_text(screen, text, font,
(255, 255, 255), constants.screen_width//2, constants.screen_height//2 - 150 +
distance * 70)
        # drawing each line of text on how to play screen
    else:
        for distance, text in enumerate(all_text[:2]):
            drawing_text(screen, text, font,
(125, 128, 0), constants.screen_width//2, constants.screen_height//2 - 50 + distance *
70)
            #only drawing the controls part of the how to play screen when in
            pause menu during a round

        for event in pygame.event.get():
            # checking if
            pressed back button or esc button
            mouse_pos = pygame.mouse.get_pos()
            if event.type == pygame.MOUSEBUTTONDOWN:
                if back_rect.collidepoint(mouse_pos):
                    running = False

            elif event.type == pygame.KEYUP:
                if event.key == pygame.K_ESCAPE:
                    running = False

            elif event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()

        pygame.display.flip()

def display_pause_menu():
    transparent_rect =
transparent_background.get_rect(center=screen.get_rect().center)
function to display pause menu with different options
    screen.blit(transparent_background, transparent_rect)
    text_list = ["Game is currently paused", "RESUME PRESS R", "CONTROLS PRESS
C", "RETURN TO MENU PRESS M"]
    y_position = constants.screen_height//2 - 150
    for text in text_list:

```

```

        drawing_text(screen,text,font,
(255,255,255),constants.screen_width//2,y_position)
        y_position += 75

# creating classes

class Gun(pygame.sprite.Sprite):
    def __init__(self,image,player):      # setting gun variables as a sprite group
        super().__init__()
        self.image = image
        self.rect = self.image.get_rect()
        self.player = player

    def update(self):
        self.rect.centerx = self.player.rect.centerx + 20      # updating the
position of gun with the position of the player
        self.rect.centery = self.player.rect.centery

    def draw(self,surface):
        surface.blit(self.image,self.rect)

class ShootingObject(pygame.sprite.Sprite):      # setting variables for this
shooting objects class as a sprite group
    def __init__(self,image,x,y):
        super().__init__()
        self.image = image
        self.rect = self.image.get_rect(center=(x,y))

class Bullets(ShootingObject):      # setting bullets as a
subclass of shooting object parent class while setting other variables
    def __init__(self,image,x,y, player):
        super().__init__(image,x,y)
        self_x = x
        self_y = y      #initialising all
the variables for the bullets
        self.player = player
        self.image = image
        self.rect = self.image.get_rect(center = (self_x,self_y))
        self.speed = 7
        self.previous_time = 0

    pos_x , pos_y = pygame.mouse.get_pos()
    self.distance_x = pos_x - self.rect.centerx      #calculating angle
between bullets and mouse
    self.distance_y = pos_y - self.rect.centery
    self.angle = math.atan2(self.distance_y,self.distance_x)

    player_pos_x = player.rect.centerx
    player_pos_y = player.rect.centery
    self.distance_x = player_pos_x - self.rect.centerx
    self.distance_y = player_pos_y - self.rect.centery
    self.enemy_angle = math.atan2(self.distance_y,self.distance_x)

```



```

# calculating angle between enemy and player

def enemy_shooting(self):

    move_x = math.cos(self.enemy_angle) * self.speed          # controls
movement of enemy bullets
    move_y = math.sin(self.enemy_angle) * self.speed
    self.rect.centerx += move_x
    self.rect.centery += move_y

def update(self, enemy_list, surface, damage_text, kills):

    player_kills = kills          # storing players
kill score

    move_x = math.cos(self.angle) * self.speed
    move_y = math.sin(self.angle) * self.speed

    self.rect.centerx += move_x
    self.rect.centery += move_y          # controlling player
movement

    for enemy in enemy_list:
        if enemy.rect.colliderect(self.rect):
            enemy.health -= 30
            damage_text_timer = 2000          #
checking for collision of bullets with enemies and enemies lose health if so. also
displaying damage text with a cooldown
            time_passed = pygame.time.get_ticks() - self.previous_time
            if enemy.health > 0:
                if damage_text_timer > 0:
                    surface.blit(damage_text, (enemy.rect.x, enemy.rect.y))
                    damage_text_timer -= time_passed

                    self.previous_time = pygame.time.get_ticks()
                elif enemy.health <= 0:
                    enemy_list.remove(enemy)          # if enemy dies they
are removed from the enemy list and player kills recorded is incremented. bullet
is removed from bullet sprite group when collision
                    player_kills += 1
                    break
            self.kill()

    if self.rect.right < 0 or self.rect.left > constants.screen_width or
self.rect.top > constants.screen_height or self.rect.bottom < 0:          # if
the bullet goes off the screen window it is deleted from sprite group
        self.kill()

    return player_kills

```

```

def draw(self,surface):
    surface.blit(self.image,self.rect)                # draws bullets on screen

class Characters(pygame.sprite.Sprite):
    last_assigned_id = 0
    def __init__(self,x,y,character,character_list,health):
        super().__init__()
        self.id = Characters.last_assigned_id # assigning each character with
unique id
        Characters.last_assigned_id += 1
        self.x = x
        self.y = y                                     #
initialising all the variables for the characters
        self.turn = False
        self.rect = pygame.Rect(x,y,50,50)
        self.health = health
        self.speed = 1.5
        self.scroll = [0,0]
        self.radius = 200
        self.character = character
        self.character_list = character_list
        self.character_animations = self.character_list[self.character]
        self.current_animation = 0
        self.animation_index = 0
        self.image = self.character_animations[self.current_animation]
[self.animation_index]
        self.previous_time1 = 0
        self.previous_time2 = 0
        self.previous_time3 = 0
        self.animation_cooldown = 0.15

def move(self,change_x,change_y):

    self.rect.x += change_x        # updating player movement
    self.rect.y += change_y

    if change_x < 0:                # turning the sprite when change direction
        self.turn = True
    if change_x > 0:
        self.turn = False

    if change_x != 0:                #checks if player is moving or
idle and based on that sets the animation state to running or idle
        self.current_animation = 1
    elif change_y != 0:
        self.current_animation = 1
    else:
        self.current_animation = 0

```

```

        if self.character != 0:
            self.rect.x += self.scroll[0] # making the enemies move by the scroll
value

        if self.character == 0:
            if self.rect.right >= constants.screen_width - self.radius:
# updating the scroll value based on the players movement and position
                self.scroll[0] = constants.screen_width - self.radius -
self.rect.right
            elif self.rect.left <= self.radius:
                self.scroll[0] = self.radius - self.rect.left

            if self.rect.right > constants.screen_width: #
setting the boundaries of the top and bottom of the map for the player only
                self.rect.right = constants.screen_width
            if self.rect.left < 0:
                self.rect.left = 0
            if self.rect.top < 40:
                self.rect.top = 40
            if self.rect.bottom > constants.screen_height - 50:
                self.rect.bottom = constants.screen_height - 50

        return self.scroll

def collision(self, enemy_list, health_pack_list):
    cooldown = 0.2
    for enemy in enemy_list:
        if enemy.character != 2:
            if self.rect.colliderect(enemy.rect):
                time_now = pygame.time.get_ticks()
                elapsed_time = (time_now - self.previous_time2) / 1000
#iterating through enemy list to check if it is colliding with the player and if
it is player takes damage with a cooldown
                if self.health >= 0 and elapsed_time > cooldown:
                    self.damage = 3
                    self.health -= self.damage
                    self.previous_time2 = time_now

    for health_pack_rect in health_pack_list:
        if health_pack_rect.colliderect(self.rect) and self.health < 100:
# iterates through health pack list and checks if player collides with healthpack
and gives 20 health if so and removes from list
            health_pack_list.remove(health_pack_rect)
            self.health += 20

    return self.health

def collision_enemies(self, enemy_list):
    if self.character != 2:
        cooldown = 1
        current_time = pygame.time.get_ticks() / 1000
        for enemy in enemy_list:

```

```

        if enemy != self:
            if self.rect.colliderect(enemy.rect) and (current_time -
self.previous_time1) > cooldown:          #apart from char 2 enemies colliding with
eachother to move slightly in a random direction to avoid them from stacking up in
one spot with a cooldown
                self.rect.x += random.randint(-10,10)
                self.rect.y += random.randint(-10,10)
                self.previous_time1 = current_time

def follow_player(self,player):

    distance_x = player.rect.x - self.rect.x
    distance_y = player.rect.y - self.rect.y
    angle = math.atan2(distance_y,distance_x)          # calculating
angle between enemies and player
    self.distance = math.sqrt(distance_y ** 2 + distance_x ** 2)    #
calculating the distance between the current enemy and the player

    if self.character != 2:
        if self.distance > 50:
            move_x = math.cos(angle) * self.speed
            move_y = math.sin(angle) * self.speed          # calculates
moving vector of enemy
            self.move(move_x,move_y)          # enemies move towards
player 50 radius
        if self.character == 3:          # setting char
3's speed more than others
            self.speed = 3
        else:
            self.current_animation = 0          # sets the
animation to zero otherwise when it is within that range which is the idle and
attacking animation
        else:
            if self.distance > 300:
                move_x = math.cos(angle) * self.speed
                move_y = math.sin(angle) * self.speed          # enemy 2
radius 300 controls movement before within radius
                self.move(move_x,move_y)
            else:
                self.current_animation = 0

def draw(self,surface):
    if ((pygame.time.get_ticks() / 1000) - self.previous_time3) >
self.animation_cooldown:
        self.animation_index = (self.animation_index + 1) %
len(self.character_animations[self.current_animation])    # increments the
animation index so that the character animation changes state from running to
idle/attack
        self.previous_time3 = pygame.time.get_ticks() / 1000
# mods it so it loops through the same animations and changes based on if its
running or idle and is controlled by cooldown

```

```

        self.image = self.character_animations[self.current_animation]
        [self.animation_index]
        player_image = pygame.transform.flip(self.image,self.turn,False)
# turning the sprite
        surface.blit(player_image,self.rect)
# draws player/enemy with rect and changing animation state and type

#main entering game playing function

def main(username):

    #initialising game

    font = pygame.font.Font(None,75) #setting the different font sizes
    font1 = pygame.font.Font(None,30)

    small_background = pygame.image.load("images/background.jpg").convert_alpha()
    background = pygame.transform.scale(small_background,(constants.screen_width,
constants.screen_height)) # loading background in correct size

    wall_img = pygame.image.load("images/map/wall.png").convert_alpha() #
loading the tile map tile images
    wall_img = pygame.transform.scale(wall_img,(40,40))
    floor_img = pygame.image.load("images/map/floor.png").convert_alpha()
    floor_img =pygame.transform.scale(floor_img,(40,40))
    dirt_img = pygame.image.load("images/map/dirt.png").convert_alpha()
    dirt_img =pygame.transform.scale(dirt_img,(40,40))
    step_img = pygame.image.load("images/map/step.png").convert_alpha()
    step_img =pygame.transform.scale(step_img,(40,40))

    player_image =
pygame.image.load("images/characters/player/idle/0.png").convert_alpha() #
loading player image in correct size
    player_image = pygame.transform.scale(player_image,(60,60))

    unscaled_gun = pygame.image.load("images/gun.png").convert_alpha() #
loading player and enemy bullets/gun in correct size
    gun_image = pygame.transform.scale(unscaled_gun,(50,70))
    unscaled_bullet = pygame.image.load("images/bullet.png").convert_alpha()
    bullet_image = pygame.transform.scale(unscaled_bullet,(15,15))
    enemy_bullet = pygame.image.load("images/enemybullet.png").convert_alpha()
    enemy_bullet_image = pygame.transform.scale(enemy_bullet,(15,15))
    damage_text = font1.render("-10",True,(255,0,0)) # damage text when enemy
takes damage

    skull = pygame.image.load("images/skull.png").convert_alpha() #
loading skull images for game over screen to store in a list

```

```

skull_image = pygame.transform.scale(skull,(100,80))
skull_list = []

health_pack = pygame.image.load("images/health.png").convert_alpha() #
loading health pack images for health and setting up variables for this function
health_pack_image = pygame.transform.scale(health_pack,(50,50))
range_x = (80,constants.screen_width-80)
range_y = (80,constants.screen_height-80)
health_pack_list = []
health_spawn_time = pygame.time.get_ticks()
health_pack_cooldown = random.randint(15000,20000)

powerup_image = pygame.image.load("images/powerup.png").convert_alpha() #
loading powerup images and setting up variables for this function
powerup_image = pygame.transform.scale(powerup_image,(50,50))
powerup_type1 = []
powerup_type2 = []
powerup_queue = [powerup_type1,powerup_type2] # putting power ups into a
queue so that it can be popped out when a power up spawns
powerup_index = 0
powerup_spawn_time = pygame.time.get_ticks()
powerup_cooldown = random.randint(18000,25000)

different_characters = ["player","enemy1","enemy2","enemy3"] # nested list
structure to store different image frames and types for each character for
animation
animation_types = ["idle","run"]
character_list = []

# final list stores type of character with index of idle or run then another
index of which frame
for character in different_characters:
    animation_list = []
    for animation in animation_types:
        temp_list = []
        for i in range(4):
            char_image =
pygame.image.load(f"images/characters/{character}/{animation}/{i}.png").convert_al
pha()
            temp_list.append(char_image)
            animation_list.append(temp_list)
        character_list.append(animation_list)

# creating in game functions

def creating_enemies(num_enemies, character_list, different_characters):
# creating instances of different enemies and appending them to the enemy list
    range_x = (constants.screen_width, 3000) # range for where the enemies can
spawn between outside the screen frame and 3000 x and y
    range_y = (constants.screen_height, 3000)
    enemy_type = 1
    enemy_list = []
    for num in range(0, num_enemies): # looping through how many

```

```

enemies there are and creating an instance of them and appending them to the enemy
list
    if enemy_type == 1:                                # controls which enemies have
more health
        health = 180
    else:
        health = 100
    enemy = Characters(random.randint(*range_x), random.randint(*range_y),
enemy_type, character_list,health) # instantiating enemies and appending them to
list
    enemy_list.append(enemy)
    enemy_type += 1                                     # increment enemy type to change the type
of enemy that spawns by one for each enemy out of the 3 types
    if enemy_type > len(different_characters) - 1:      # loops back to
first enemy type so that it doesnt go over the max and cause index error
        enemy_type = 1
    return enemy_list    # return the list of enemies currently alive to be
used in code

num_enemies = 5
# setting the number of enemies at the start of the game and using the function to
create them
    enemy_list = creating_enemies(num_enemies, character_list,
different_characters)

def enemy_shooting_mechs(max_bullets):
# function for mechanics of the enemy 3's shooting
    enemy_bullet_cooldowns = {}    # create dictionary for cooldown and
previous shot
    enemy_bullet_previous_times = {}

    for enemy in enemy_list:
        if enemy.character == 2 and enemy.current_animation == 0: # iterate
through enemy list and check only for character 3 and if they are idle
            if enemy.id not in enemy_bullet_cooldowns:                #if the
current enemy is not present in the cooldown dictionary it sets the initial
cooldown to 2
                enemy_bullet_cooldowns[enemy.id] = 2

                if enemy.id not in enemy_bullet_previous_times:        # if
current enemies previous shot is not in previous times dictionary it sets it to 0
                    enemy_bullet_previous_times[enemy.id] = 0

                    enemy_time_passed = (pygame.time.get_ticks() -
enemy_bullet_previous_times[enemy.id]) / 1000    # checks the time passed
                    if len(enemy_list) > 5:
                        max_bullets += 5
                    if max_bullets > 100:
                        max_bullets = 100 # sets the maximum amount of bullets
shoot at once on the screen

                    if enemy_time_passed > enemy_bullet_cooldowns[enemy.id] and
len(enemy_bullet_group) < max_bullets:                # if the time passed is

```

greater than cooldowns and less than max bullets then it will create an instance of a bullet

```

        bullet = Bullets(enemy_bullet_image, enemy.rect.centerx + 10,
enemy.rect.centery, player)
        enemy_bullet_group.add(bullet)
        enemy_bullet_previous_times[enemy.id] =
pygame.time.get_ticks() # resets previous times
        enemy_bullet_cooldowns[enemy.id] = 2
# enemy.id allows for a unique identifier for each enemy in enemy_list so each
character 3 will have a seperate bullet cooldown

```

```

def health_bar(surface,health): # health bar function to player
health display
    full_health = 100
    remaining_health = min(health,full_health)
    ratio = remaining_health/full_health
    rect = pygame.draw.rect(surface,(255,0,0),(constants.screen_width - 350,
20,300,45))
    health_length = int(rect.width*ratio)
    pygame.draw.rect(surface,(0,128,0),(constants.screen_width - 350,
20,health_length,45)) # health is drawn based on the ratio, red rect is drawn
over the green

    return health_length

```

```

def map(scroll): # function for drawing each
tile on the screen and moving it by the scroll value when the player moves based
on the map list
    y = 0
    for row in map_list.game_map:
        x = 0
        for tile in row:
            tile_rect = pygame.Rect(x * tile_size + scroll[0], y * tile_size +
scroll[1], tile_size, tile_size)
            if tile == '1':
                screen.blit(floor_img, (x*tile_size + scroll[0], y*tile_size +
scroll[1]))
            if tile == '2':
                screen.blit(wall_img, (x*tile_size + scroll[0], y*tile_size +
scroll[1]))
                check_tile_collision(tile_rect)
            if tile == '3':
                screen.blit(dirt_img, (x*tile_size + scroll[0], y*tile_size +
scroll[1]))
            if tile == '4':
                screen.blit(step_img, (x*tile_size + scroll[0], y*tile_size +
scroll[1]))
            if tile != '0':
                tile_rects.append(pygame.Rect(x * tile_size + scroll[0],
y*tile_size + scroll[1], tile_size, tile_size))
                x += 1
        y += 1

```



```
with open("map.txt", "r") as file:                                # reads the map list file
    game_map = [list(line.strip()) for line in file]

def check_tile_collision(tile_rect):
    #checking player collision with the wall tile on the left and right of the
    map so it cant go through
    if player.rect.colliderect(tile_rect):
        if change_x > 0:
            player.rect.right = tile_rect.left
        if change_x < 0:
            player.rect.left = tile_rect.right

# setting variables

time_now = time.time()                                           # setting initial variables for the
start of the main loop
start_time = time.time()

kills = 0                                                         # initial and constant variables
health = 100
waves = 0
tile_size = 40
enemy_max_bullets = 20

previous_time = 0                                                 # timer initial variables
game_over_timer = 0
bullet_cooldown = 0.5
powerup_duration = 0
paused_time = 0
wave_text_duration = 3000
wave_text_time = 0
powerup_text_duration = 3000
powerup_text_time = 0

moving_up = False                                                 # initial movement status
moving_down = False
moving_left = False
moving_right = False

wave_status = False                                               # inital boolean status
powerup1_text = False
powerup2_text = False

paused = False                                                    # initially sets run for main loop as true and game
over as false and pause menu as false
game_over = False
run = True

player = Characters(700,300, 0, character_list,health)           # creating
instances of player and gun
gun = Gun(gun_image,player)
```

```

bullet_group = pygame.sprite.Group()      # creating sprite groups to store
bullets for player and enemy
enemy_bullet_group = pygame.sprite.Group()

# game loop

while run:

    for event in pygame.event.get():
        if event.type == pygame.QUIT:      # if user presses cross quits the
program
            pygame.quit()
            sys.exit()

        if event.type == pygame.KEYDOWN:    # movement mechanics for player
checks each button wasd for input if so sets moving to true otherwise false
            if event.key == pygame.K_w:
                moving_up = True
            if event.key == pygame.K_s:
                moving_down = True
            if event.key == pygame.K_a:
                moving_left = True
            if event.key == pygame.K_d:
                moving_right = True

        if event.type == pygame.KEYUP:
            if event.key == pygame.K_w:
                moving_up = False
            if event.key == pygame.K_s:
                moving_down = False
            if event.key == pygame.K_a:
                moving_left = False
            if event.key == pygame.K_d:
                moving_right = False

        mouse_pos = pygame.mouse.get_pos()
        bullet_time_passed = (pygame.time.get_ticks() - previous_time) / 1000
# setting cooldown
        if event.type == pygame.MOUSEBUTTONDOWN and bullet_time_passed >
bullet_cooldown:      # checks if user left click
            if not pause_rect.collidepoint(mouse_pos) and event.button == 1:
# and if its not pressing the pause button
                bullet = Bullets(bullet_image,player.rect.centerx + 50,
player.rect.centery, player)      # creates instance of a bullet and adds it to the
sprite group
                bullet_group.add(bullet)
                previous_time = pygame.time.get_ticks()      # resets cooldown

        if event.type == pygame.MOUSEBUTTONDOWN:    # checking if pressing
pause button
            mouse_pos = pygame.mouse.get_pos()

```

```

        if pause_text_rect.collidepoint(mouse_pos):
            paused = True

    if paused:
        start_paused_time = time.time()          # if the game is paused it
calculates paused time to take away from actual in game time
        seconds = start_paused_time - time_now
        paused_time = int(seconds)

        display_pause_menu()          # call display function to display
pause menu screen
        if event.type == pygame.KEYUP:          # for each option on the menu
screen, if its letter is pressed it does that function
            if event.key == pygame.K_r: # r resumes the game
                paused = False
                time_now = time.time()
            elif event.key == pygame.K_m:  # m ends game and returns back to
menu
                menu()
                run = False
                paused = False
            elif event.key == pygame.K_c:  # c shows controls screen
                controls(None)
            elif event.key == pygame.K_ESCAPE:  # if esc pressed you can also
resume
                paused = False

        pygame.display.update()
        clock.tick(60)

    else:  # otherwise continue with the rest of the game

        time_now = time.time()
        seconds = time_now - start_time
        elapsed_time = int(seconds) - paused_time  # calculating the time
survived in the game

        screen.fill((0,0,0))  # setting black fill for display update and
setting background
        screen.blit(background,(0,0))

        change_x = 0
        change_y = 0          # handling movement change and speed
        if moving_right == True:
            change_x = 2.5
        if moving_left == True:
            change_x = -2.5
        if moving_up == True:
            change_y = -2.5
        if moving_down == True:
            change_y = 2.5

```

```

    tile_rects = []      # setting the list for the rects of all tiles
    scroll = player.move(change_x,change_y) # calling player move function
and receiving the scroll function to pass it into the map function
    map(scroll) # creates and controls the map scroll

# managing the health packs
current_time_health = pygame.time.get_ticks()

if current_time_health - health_spawn_time >= health_pack_cooldown: #
spawn cooldown
    health_spawn_time = current_time_health

    health_x = random.randint(*range_x)
    health_y = random.randint(*range_y)    # health spawn range
    health_pack_rect = pygame.Rect(health_x,health_y,50,50)
    if len(health_pack_list) <= 1:
        health_pack_list.append(health_pack_rect) # not more than 2
at a time

    for health_pack_rect in health_pack_list:
        screen.blit(health_pack_image,health_pack_rect) # drawing each
health pack in the list

# managing the power ups
current_time_powerup = pygame.time.get_ticks()
powerup_list = powerup_queue[powerup_index]    # picking the current
power up from the queue with a random index

if current_time_powerup - powerup_spawn_time >= powerup_cooldown: #
cooldown
    powerup_spawn_time = current_time_powerup

    powerup_x = random.randint(*range_x)
    powerup_y = random.randint(*range_y) # power up range
    powerup_rect = pygame.Rect(powerup_x,powerup_y,50,50)
    powerup_index = random.randint(0,1)    # pick a random index to
switch to a random power up from the queue
    if len(powerup_list) <= 1:
        powerup_list.append(powerup_rect) # no more than 2 at a time

    if powerup_list == powerup_type1: # code to handle the first power
up type to check collision, remove and change bullet speed
        for powerup in powerup_list:
            if player.rect.colliderect(powerup):
                powerup_list.remove(powerup)
                bullet_cooldown = 0.1
                powerup_duration = 3
                powerup1_text = True
                powerup_text_time = pygame.time.get_ticks()
            elif powerup_list == powerup_type2: # code to handle the second power

```

```

up type to check collision, remove
    for powerup in powerup_list:
        if player.rect.colliderect(powerup):
            powerup_list.remove(powerup)
            powerup_duration = 5
            powerup2_text = True
            powerup_text_time = pygame.time.get_ticks()

    if powerup1_text == True and pygame.time.get_ticks() -
powerup_text_time < powerup_text_duration:      # code to display text for both
power ups for a few secs
        drawing_text(screen, "RAPID SHOOTING 3s", font,
(255,0,255), constants.screen_width//2, constants.screen_height//2 - 100)
        elif powerup2_text == True and pygame.time.get_ticks() -
powerup_text_time < powerup_text_duration:
            drawing_text(screen, "FREEZE 5s", font,
(255,0,255), constants.screen_width//2, constants.screen_height//2 - 100)
            for enemy in enemy_list: # stops enemy movement as power up type
2 for this duration
                enemy.speed = 0
            else:
                powerup1_text = False          # otherwise game will continue as
normal
                powerup2_text = False
                player.speed = 1.5
                for enemy in enemy_list:
                    if enemy.character == 3:
                        enemy.speed = 3
                    else:
                        enemy.speed = 1.5

    if powerup_duration > 0:
        powerup_duration -= 1 / 60 # power up bullet duration
    elif powerup_duration <=0:
        bullet_cooldown = 0.5

    for powerup_rect in powerup_list:
        screen.blit(powerup_image, powerup_rect) # drawing each power up in
the list

    player.draw(screen) # drawing player gun and
bullets in list
    gun.draw(screen)
    gun.update()
    for bullet in bullet_group:
        bullet.draw(screen)
        kills = bullet.update(enemy_list, screen, damage_text, kills)

    for enemy in enemy_list: # for each enemy in
list drawing, checking collision and movement control
        enemy.follow_player(player)
        enemy.collision_enemies(enemy_list)
        enemy.draw(screen)

```

```

        enemy_shooting_mechs(enemy_max_bullets) # managing enemy bullet
control
    for bullet in enemy_bullet_group:
        bullet.enemy_shooting()
        bullet.draw(screen)
        if bullet.rect.colliderect(player.rect): #checking collision of
bullet with player
            if player.health >= 0:
                player.health -= 0.5
                enemy_bullet_group.remove(bullet)

            if bullet.rect.right < 0 or bullet.rect.left >
constants.screen_width or bullet.rect.top > constants.screen_height or
bullet.rect.bottom < 0:
                enemy_bullet_group.remove(bullet) # if the bullet goes off the
game screen it is removed from the list

        if enemy_list == []: # if all the enemies are killed new instance
of enemies created but with 1 more enemy each time, wave value incremented
            num_enemies += 1
            waves += 1
            wave_status = True
            wave_text_time = pygame.time.get_ticks()
            enemy_list = creating_enemies(num_enemies,character_list,
different_characters)

        if current_username != None: # if the user is logged
in stats updated if needed
            highest_time_survived = elapsed_time
            update_stats(username, kills, waves, highest_time_survived)

        if wave_status and pygame.time.get_ticks() - wave_text_time <
wave_text_duration: # with a cooldown wave text appears every new wave
            drawing_text(screen, f"Wave {waves}", font, (0,0,0),
constants.screen_width // 2,constants.screen_height // 2)

        health_bar(screen,health)
# in game UI features are drawn
        drawing_text(screen,f' Kills: {kills}',font1,(255, 0, 0), 250, 50)
        pause_rect = pygame.Rect(50, 30, 100, 30)
        pause_text_rect = drawing_text(screen,"Pause",font1,(0,0,0),None,None,
(128,128,128),pause_rect)

        health = player.collision(enemy_list,health_pack_list) # checking
health when collisions occur
        if health <= 0: # if it reaches
zero game stops running and game over screen appears
            run = False
            game_over = True

        pygame.display.update() # updating the display and setting the fps
        clock.tick(60)

```

```

# game over screen

while game_over == True:
    screen.fill((0,0,0))
    game_over_timer += clock.get_time()    # timer for skull spawn

    current_time_skull = pygame.time.get_ticks()
    skull_cooldown = 200
    skull_spawn_time = current_time_skull - previous_time

    if skull_spawn_time >= skull_cooldown and game_over_timer < 10000: #
cooldown for skull append to list
        skull_x = random.randint(*range_x)# skull range
        skull_y = random.randint(*range_y)
        skull_rect = pygame.Rect(skull_x, skull_y, 50, 50)
        skull_list.append(skull_rect)
        previous_time = current_time_skull

    for skull_rect in skull_list:
        screen.blit(skull_image,skull_rect)    # drawing each skull in list

    player_image = pygame.transform.scale(player_image,(60,80))
# player image and text to draw on screen
    screen.blit(player_image,(constants.screen_width//2-
50,constants.screen_height//2-50))
    drawing_text(screen,"Game Over",font,
(255,0,0),constants.screen_width//2,constants.screen_height//2-100)
    drawing_text(screen,"Press SPACE to play again",font,
(0,0,255),constants.screen_width//2,constants.screen_height//2+100)
    drawing_text(screen,f"Total kills: {kills}",font,
(0,0,225),constants.screen_width//2,constants.screen_height//2+200)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.KEYDOWN:    #checking space bar to send
back to menu and retry
            if event.key == pygame.K_SPACE:
                game_over = False

    pygame.display.update()    # updating the display and setting the fps
    clock.tick(60)

# menu UI functions

def login():
    global current_username    # setting username as a global variable so it can be
used in the main code and elsewhere

    while True:
        username = input("Enter username: ")    # checks user login if
correct logs in successfully else asks you if you want to create an account

```

```

setting the username as current username
    if username == '':
        print("Exited")
        return
    if check_player_username(username):
        print("User found.")
        password = input("Enter password: ")
        if password == '':
            print("Exited")
            return
        if check_player_password(username, password):
            print("Successfully logged in as",username)
            current_username = username # sets global
variable current username to the username entered
            break
        else:
            print("Incorrect password, try again or enter empty space to exit.
")
    else:
        print("User not found. Would you like to create an account? ")
        response = input("Enter 'yes' to create a new account or 'no' to try
again or enter empty space to exit: ") # must follow validity checks to
successfully login
        if response.lower() == 'yes':
            create_account()
            break
        elif response == 'no':
            continue
        elif response == '':
            print("Exited")
            return
        else:
            print("Invalid input. please put 'yes' or 'no' or enter empty. Now
try logging in again. ")

def create_account():
    while True:
        username = input("Enter username: ") # validity checks for username
and password when creating an account

        if username == '':
            print("Exited")
            return
        if len(username) <= 1:
            print("Username is too short. Must be between 1 - 10 characters. Enter
empty space to exit.")
            continue
        elif len(username) >= 10:
            print("Username is too long. Must be between 1 - 10 characters. Enter
empty space to exit.")
            continue

```



```

        password = input("Enter password: ")
        if password == '':
            print("Exited")
            return
        if len(password) <= 1:
            print("Password is too short. Must be between 1 - 10 characters. Enter empty space to exit.")
            continue
        elif len(password) >= 10:
            print("Password is too long. Must be between 1 - 10 characters. Enter empty space to exit.")
            continue

        if not any(char.isdigit() for char in password):
            print("Password must contain at least one number.")
            continue

        if check_player_username(username):
            valid = False
            while not valid:
                response = input("Username already exists. Do you want to try again? (yes/no): ").lower()
                # if username already exists asks if you want to try again or create an account
                if response == 'yes':
                    break
                elif response == 'no':
                    return
                else:
                    print("Invalid input. Please enter 'yes' or 'no'.")
                    continue

            else:
                score = 0
                waves = 0
                time = 0
                add_player(username, password, score, waves, time)
                # setting initial player stats if account successfully created and creating account in database
                print("Account created successfully. Now log in")
                return

def menu():
    screen.fill((0,0,0))

    background1 = pygame.image.load("images/background2.jpg").convert_alpha()
    background1 = pygame.transform.scale(background1,
    (constants.screen_width, constants.screen_height))
    # setting background for menu
    screen.blit((background1), (0,0))
    font = pygame.font.Font(None, 30)

    selection = [{"button rect": pygame.Rect(constants.screen_width // 2 - 100,
    constants.screen_height - 650, 200, 50), "writing": "PLAY"},
    #

```

setting a list for the different buttons to be blitted in the same format but different positons

```

        {"button rect": pygame.Rect(constants.screen_width // 2 - 100,
constants.screen_height - 550 ,200,50), "writing": "LOGIN"},
        {"button rect": pygame.Rect(constants.screen_width // 2 - 100,
constants.screen_height - 450 ,200,50), "writing": "CREATE ACCOUNT"},
        {"button rect": pygame.Rect(constants.screen_width // 2 - 100,
constants.screen_height - 350 ,200,50), "writing": "LEADERBOARD"},
        {"button rect": pygame.Rect(constants.screen_width // 2 - 100,
constants.screen_height - 250 ,200,50), "writing": "HOW TO PLAY"},
        {"button rect": pygame.Rect(constants.screen_width // 2 - 100,
constants.screen_height - 150 ,200,50), "writing": "QUIT"]}

```

```

    for selector in selection:
        pygame.draw.rect(screen,(128,128,128), selector["button rect"])
# drawing each button and the text
        writing = font.render(selector["writing"], True, (0,0,0))
        writing_rect = writing.get_rect(center=selector["button rect"].center)
        screen.blit(writing, writing_rect)

    if current_username != None:
        user_rect = pygame.Rect(constants.screen_width -275,
constants.screen_height -50 ,100,30)          # if user is logged in text will show
saying you are logged in with your account name
        user = font.render("Logged in as: "+str(current_username), True,
(225,215,0))
        user_rect = user.get_rect(center=user_rect.center)
        screen.blit(user,user_rect)
    else:
        user_rect = pygame.Rect(constants.screen_width -275,
constants.screen_height -50 ,100,30)          # if user is not logged in it will say
you are not logged in
        user = font.render("You are not logged in", True, (255,0,0))
        user_rect = user.get_rect(center=user_rect.center)
        screen.blit(user,user_rect)

```

```

running = True
while running:
    for event in pygame.event.get():

        if event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1:
                mouse_pos = pygame.mouse.get_pos()          #
using the buttons list format to return each button based on which is pressed to
the initial code
                for selector in selection:
                    if selector["button rect"].collidepoint(mouse_pos):
                        if selector["writing"] == "LOGIN":
                            return "LOGIN"
                        elif selector["writing"] == "PLAY":
                            return "PLAY"
                        elif selector["writing"] == "CREATE ACCOUNT":
                            return "CREATE ACCOUNT"

```

```
        elif selector["writing"] == "LEADERBOARD":
            return "LEADERBOARD"
        elif selector["writing"] == "HOW TO PLAY":
            return "HOW TO PLAY"
        elif selector["writing"] == "QUIT":
            return "QUIT"

        if event.type == pygame.QUIT:                                # quits if crosses
game
            pygame.quit()
            sys.exit()

        pygame.display.flip()

# start code

game_over = False
game = True                # initialising the game to true to always run the
                           # menu at the start and lead to the other options
while game:
    menu_option = menu()
    if menu_option == "PLAY":                # based on what was returned to the menu
when a button is pressed it will execute that desired function
        main(current_username)              # play when user not logged in username is
none so no data is saved but if logged in username is passed to main function so
it will be updated during the game
    elif menu_option == "CREATE ACCOUNT":
        create_account()
    elif menu_option == "LEADERBOARD":
        leaderboard()
    elif menu_option == "LOGIN":
        login()
    elif menu_option == "HOW TO PLAY":
        controls(0) # 0 passed in so that argument is not None
    elif menu_option == "QUIT":
        pygame.quit()
        sys.exit()

pygame.quit() # quits if player crosses
sys.exit() # exits system
```